
EVE-Agent: Evidence-Verifiable Self-Evolving Agents

Yamato Arai
Fujitsu Limited
Department of Basic Science
The University of Tokyo

Yuma Ichikawa
Fujitsu Limited
RIKEN center for AIP

Abstract

Self-evolving agents should not train on examples they cannot justify. Data-free self-evolving search agents offer a scalable route to systems that generate their own questions, answer them, and improve from their own feedback without human annotations. Yet, without verifiable evidence, this loop can reward fluent but unsupported examples, turning the self-generated curriculum into an opaque and potentially unreliable training signal. We argue that evidence verifiability is a prerequisite for trustworthy self-evolution in search agents: each generated instance should include not only an answer but also a source-grounded span whose contribution to that answer can be measured. We introduce EVE-Agent, an Evidence-Verifiable Self-Evolving Agent that operationalizes this principle through a modification to the proposer–solver framework. The proposer generates a question, an answer, and a verbatim evidence span. An evidence verifier then rewards the span according to the marginal accuracy gain when the evidence is provided. This produces a training signal that favors evidence that genuinely helps answer the question, without requiring oracle answers, human labels, or external annotations. EVE-Agent leaves the backbone model, retriever, search tool, and optimization framework unchanged. Experiments show that EVE-Agent substantially improves evidence-grounded correctness over prior self-evolving search agents. The resulting curriculum is not merely self-generated but auditable by construction: each training example carries an inspectable source span that explains why it should be trusted.

1 Introduction

Search agents for knowledge-intensive question answering must do more than retrieve relevant information: they must also ground their answers in appropriate evidence. This requirement distinguishes them from standard language models, which may generate fluent responses without revealing why those responses should be trusted. In supervised settings, evidence grounding is commonly enforced using human-curated question–answer datasets annotated with supporting spans, such as HotpotQA, 2WikiMultiHopQA, and MuSiQue [Yang et al., 2018, Ho et al., 2020, Trivedi et al., 2022]. Retrieval-augmented and tool-using language models operationalize this evidence-seeking behavior [Lewis et al., 2020, Yao et al., 2023, Schick et al., 2023, Trivedi et al., 2023, Asai et al., 2024], while recent reinforcement-learning methods further demonstrate that models can learn to invoke search as part of their reasoning process [Jin et al., 2025, Song et al., 2025].

However, constructing evidence-grounded supervision at scale is costly, tightly coupled to a particular corpus, and difficult to update when the retrieval environment changes. Data-free self-evolution provides an attractive alternative: a model generates its own training questions, attempts to solve them, and improves from the resulting feedback. This paradigm has shown strong promise in domains such as reasoning and code, where self-generated tasks can be validated by external oracles, including interpreters and symbolic checkers [Zhao et al., 2025, Huang et al., 2026]. It has also recently been extended to multi-turn search agents [Yue et al., 2026]. However, search-based question answering

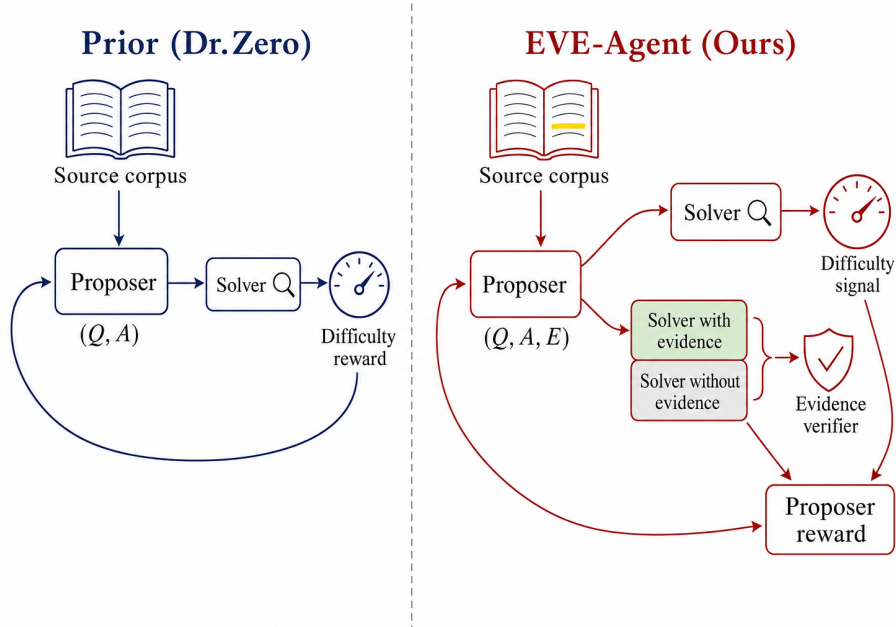


Figure 1: **Evidence-verifiable self-evolving search agents.** Existing self-evolving search agents (*left*) reward proposers using only a difficulty signal based on solver accuracy, without auditing the source evidence behind each question. EVE-Agent (*right*) requires the proposer to output a source-grounded evidence span and rewards it only when that evidence causally improves the solver’s answer accuracy, measured by the gain from no-evidence to with-evidence rollouts. This modification is limited to the reward: the proposer, solver, backbone model, and search tool remain unchanged.

lacks the exact verification mechanisms available in code or mathematics. A self-generated question may be ambiguous, unsupported by the source text, or answerable from the model’s memorized knowledge alone. Likewise, a solver may produce a confident answer without supplying evidence that genuinely supports it.

This limitation exposes a central weakness in existing self-evolving search-agent loops. Their reward signals primarily assess whether a generated question is useful as a difficulty-controlled training instance. However, they do not verify whether the corresponding answer is grounded in a source span that can be checked. Consequently, unsupported examples may enter the self-generated curriculum and shape subsequent learning. The problem is not simply that evidence may be missing. In practice, a system may produce a syntactically valid evidence block even when the cited span does not actually justify the answer. Such examples are difficult to audit: once they are incorporated into the curriculum, it becomes unclear whether the agent is learning to search for and reason over evidence, or merely reinforcing fluent but unverifiable behavior.

We argue that evidence verifiability should be a core design principle for data-free, self-evolving search agents. Each generated training instance should include a source-grounded evidence span, and the utility of that span should be explicitly measurable. This requirement reframes evidence from an optional explanation into a training-time object that can be inspected, scored, and reused. It also makes the generated curriculum more trustworthy: every question–answer pair is paired with a concrete textual basis, and the system can be evaluated not only on whether it answers correctly but also on whether it provides evidence that supports its answer. To this end, we introduce EVE-Agent, a lightweight extension of the proposer–solver framework built around evidence verifiability. The proposer generates a question, an answer, and a verbatim evidence span from the source text. An evidence verifier then rewards the proposer according to the marginal improvement in the current solver’s answer accuracy when the evidence is provided, relative to answering from the question alone. This signal requires no oracle answers, human labels, or external annotations: it is computed

solely from the solver, the proposer-emitted evidence, and the corpus. The same evidence span is subsequently used to train the solver to produce both an answer and supporting evidence. Importantly, EVE-Agent leaves the backbone model, retriever, search tool, and policy-optimization framework unchanged.

The resulting self-generated curriculum is auditable by design. Each training example is paired with an explicit source span, and that span is rewarded only when it helps the solver answer the question. This design discourages unsupported or purely memorization-based questions while preserving the scalability advantages of data-free self-evolution. It also offers a practical mechanism for inspecting the agent’s generated training data: the curriculum is no longer a collection of opaque question–answer pairs, but a set of evidence-linked instances whose grounding can be verified post hoc. Our experiments show that, under matched conditions, EVE-Agent substantially improves evidence-grounded correctness over prior self-evolving search-agent methods. These results suggest that self-evolving search agents can be trained not only to answer questions but also to produce evidence that makes their own training process verifiable.

2 Background

Notation. Let $\mathcal{D} = \{d_1, \dots, d_{|\mathcal{D}|}\}$ denote a finite corpus, where each document d_i is represented as a token sequence. A task instance is a triple (q, a, e) consisting of a question q , its target answer a , and an evidence span e . The evidence span is required to be a contiguous text span copied from either a source document in $\mathcal{D}$ or a snippet retrieved from that corpus. A search engine $\mathcal{R}$ is shared by all agents: given a text query, $\mathcal{R}$ returns a finite list of snippets drawn from $\mathcal{D}$. Throughout the paper, logarithms are natural, and $\mathbf{1}\{\cdot\}$ denotes the indicator function, which equals 1 when its argument is true and 0 otherwise. For any model M and input x , we use $M(a \mid x) := \mathbb{P}_{\hat{a} \sim M(\cdot \mid x)}[\hat{a} = a]$ to denote the probability that M generates the answer string a when conditioned on x .

Self-evolving search-agent loop. The self-evolving search-agent framework consists of two policies that are updated over training rounds $t = 1, \dots, T$. The proposer policy, denoted by π_t^{pro} , generates a training instance from a source document. In the prior framework, this instance is a question–answer pair (q, a) ; in EVE-Agent, it is extended to a question–answer–evidence triple (q, a, e) . The solver policy, denoted by π_t^{sol} , receives a question, may call the shared search engine $\mathcal{R}$ during its reasoning process, and outputs an answer. For notational convenience, we define

$$M_{\text{sol},t}(\cdot \mid x) := \pi_t^{\text{sol}}(\cdot \mid x, \mathcal{R}) \quad (1)$$

for the answer distribution induced by the solver at round t when it is given input x and access to the search engine. We use $M_{\text{pro},t}$ analogously for the distribution induced by the proposer. The setting is data-free in the sense that no human-labeled question–answer pairs or supporting spans are provided. The only human-supplied resources are the corpus $\mathcal{D}$ and the search engine $\mathcal{R}$.

Difficulty reward. We first recall the difficulty-based proposer reward used in the prior self-evolving search-agent framework [Yue et al., 2026]. At training round t , a source document $d \in \mathcal{D}$ is sampled uniformly from the corpus. The proposer then generates a question–answer pair $(q, a) \sim \pi_t^{\text{pro}}(\cdot \mid d, \mathcal{R})$, and the solver answers the same question n times independently, $\{\hat{a}_j\}_{j=1}^n \sim M_{\text{sol},t}(\cdot \mid q)$. Let $k := \sum_{j=1}^n \mathbf{1}\{\hat{a}_j = a\}$ be the number of solver predictions that exactly match the proposer-provided answer. The proposer receives the difficulty reward

$$R_t^{\text{DZ}}(q, a; k) = \mathbf{1}\{0 < k < n\} \frac{n - k}{n - 1}. \quad (2)$$

The prior system [Yue et al., 2026] also adds a format-related term to this signal; in our notation we keep R_t^{DZ} for the pure difficulty component and treat the format reward separately in Eq. (16).

This reward favors questions that are neither trivial nor impossible for the current solver. If all solver predictions are correct or all are incorrect, then the difficulty term is zero. Otherwise, the reward increases as the number of incorrect predictions grows. Thus, among questions that the solver can answer at least sometimes, the proposer is encouraged to generate examples near the solver’s current learning frontier. This intuition can be made precise. Define $p_t^{\text{sol}}(q, a) := M_{\text{sol},t}(a \mid q)$, the probability that the solver at round t generates answer string a when conditioned on question q .

Since the n solver predictions are sampled independently, the number of correct predictions follows $k \sim \text{Bin}(n, p_t^{\text{sol}}(q, a))$. Taking the expectation of Eq. (2) over this binomial randomness yields

$$\mathbb{E}_{k \sim \text{Bin}(n, p)}[R_t^{\text{DZ}}(q, a; k)] = \phi_n(p_t^{\text{sol}}(q, a)), \quad (3)$$

where

$$\phi_n(p) := \frac{n}{n-1}(1-p)(1-(1-p)^{n-1}). \quad (4)$$

Lemma B.1 proves this identity. The function ϕ_n is continuous and unimodal on $[0, 1]$, and it vanishes at both endpoints. Therefore, the expected difficulty reward is small both when the solver almost never answers correctly and when it almost always answers correctly; it is largest at an intermediate success probability.

Hop-grouped relative policy optimization. Optimizing the proposer directly with group-relative policy optimization would be costly in the search-agent setting. A direct implementation would require nested sampling: for each source document, one would sample multiple candidate questions from the proposer, and for each candidate question one would run multiple solver rollouts. Because each solver rollout may call the search engine several times, this nested procedure is prohibitively expensive.

Hop-grouped relative policy optimization (HRPO) avoids this cost by sampling one proposer output per source document while normalizing rewards within comparable groups [Yue et al., 2026]. Suppose a batch contains N generated question-answer pairs, $\{(q_i, a_i)\}_{i=1}^N$. Each question has a prescribed hop count $h_i \in \mathcal{H}$, where the hop count indicates the intended number of reasoning steps or evidence pieces needed to answer the question. For each hop value h , define $\mathcal{I}_h := \{i : h_i = h\}$.

HRPO computes the standardized advantage of example i within its hop group:

$$A_{i,h} = \frac{R_t^{\text{DZ}}(q_i, a_i; k_i) - \mathbb{E}_{j \in \mathcal{I}_h}[R_{t,j}^{\text{DZ}}]}{\sqrt{\text{Var}_{j \in \mathcal{I}_h}[R_{t,j}^{\text{DZ}}] + \delta_0}}, \quad (5)$$

where k_i is the number of correct solver predictions for example i , $R_{t,j}^{\text{DZ}}$ denotes the difficulty reward of example j , and $\delta_0 > 0$ is a numerical stabilizer.

Let $\pi_{\text{ref}}^{\text{pro}}$ be a frozen reference proposer policy. In our experiments, this reference is the proposer initialization at the beginning of Phase A. The HRPO update maximizes

$$\mathcal{J}_t^{\text{HRPO}} = \frac{1}{N} \sum_{h \in \mathcal{H}} \sum_{i \in \mathcal{I}_h} \log \pi_t^{\text{pro}}(q_i, a_i \mid d_i, \mathcal{R}) A_{i,h} - \beta \mathbb{E}_d [\text{KL}(\pi_t^{\text{pro}}(\cdot \mid d, \mathcal{R}) \parallel \pi_{\text{ref}}^{\text{pro}}(\cdot \mid d, \mathcal{R}))], \quad (6)$$

where $\beta > 0$ controls the strength of KL regularization. The KL divergence is taken between two conditional distributions over proposer outputs given the same source document d and access to the same search engine $\mathcal{R}$. Its role is to keep the current proposer π_t^{pro} close to the frozen reference $\pi_{\text{ref}}^{\text{pro}}$, thereby preventing overly large policy updates. This KL term is conceptually separate from the relative advantage normalization in Eq. (5).

Group-relative policy optimization. The solver is trained with group-relative policy optimization (GRPO) [Shao et al., 2024]. For a given question q , the rollout policy is the previous solver π_{t-1}^{sol} . It samples a group of n candidate responses, $\{\hat{y}_j\}_{j=1}^n \sim \pi_{t-1}^{\text{sol}}(\cdot \mid q, \mathcal{R})$. Each response receives a binary answer reward $r_j = \mathbf{1}\{\hat{y}_j = a\}$, where a is the target answer. Let $\bar{r}$ and $\hat{\sigma}$ be the empirical mean and standard deviation of $\{r_j\}_{j=1}^n$ within the group. The standardized advantage is $A_j = r_j - \bar{r} / \hat{\sigma} + \delta_0$, where $\delta_0 > 0$ prevents division by zero.

Let $\pi_{\text{ref}}^{\text{sol}}$ be a frozen reference solver policy. In our experiments, this reference is the solver initialization at the beginning of Phase B. GRPO maximizes the clipped surrogate

$$\mathcal{J}_t^{\text{GRPO}} = \mathbb{E} \left[\frac{1}{n} \sum_{j=1}^n \min(\rho_j A_j, \text{clip}(\rho_j, 1 - \epsilon, 1 + \epsilon) A_j) \right] - \beta \mathbb{E}_q [\text{KL}(\pi_t^{\text{sol}}(\cdot \mid q, \mathcal{R}) \parallel \pi_{\text{ref}}^{\text{sol}}(\cdot \mid q, \mathcal{R}))], \quad \rho_j = \frac{\pi_t^{\text{sol}}(\hat{y}_j \mid q, \mathcal{R})}{\pi_{t-1}^{\text{sol}}(\hat{y}_j \mid q, \mathcal{R})}, \quad (7)$$

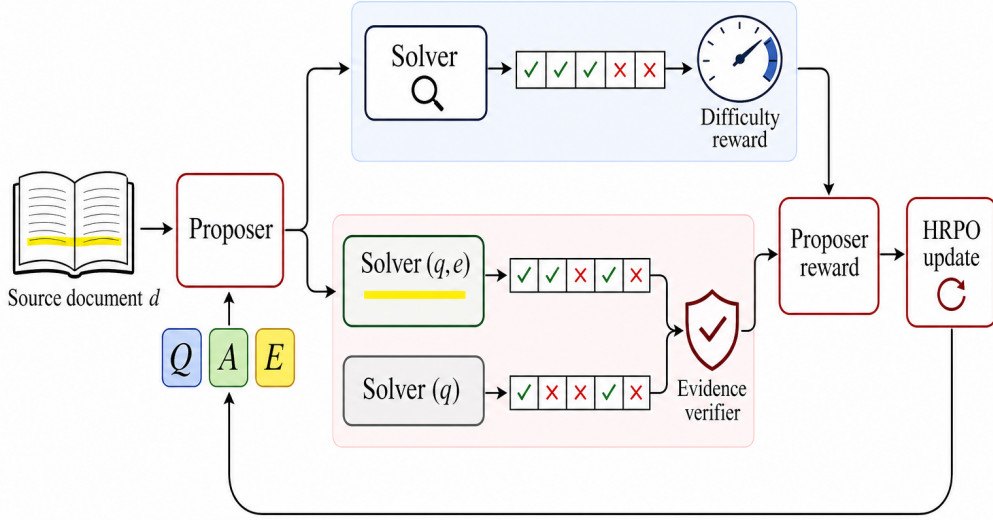


Figure 2: **One Phase A iteration of EVE-Agent.** The proposer generates a question–answer–evidence triple from the source document d . The solver attempts the question with the search tool, producing the difficulty reward of Eq. (2); in parallel, single-turn search-disabled rollouts of the solver with and without the evidence span produce the evidence verifier of Eq. (11). These two signals combine with the format and brevity terms in the proposer reward of Eq. (16), which drives one HRPO update of the proposer. The backbone, the retriever, and the search tool are unchanged.

where $\epsilon \in (0, 1)$ is the clipping width, and $\beta > 0$ is the KL coefficient.

The importance ratio ρ_j compares the probability of the sampled response $\hat{y}_j$ under the current solver π_t^{sol} with its probability under the rollout policy π_{t-1}^{sol} . In contrast, the KL term compares the full conditional output distribution of the current solver with the frozen reference solver $\pi_{\text{ref}}^{\text{sol}}$ for the same question q and search engine $\mathcal{R}$. The ratio controls the policy-gradient update on sampled responses, while the KL term regularizes the entire updated policy. EVE-Agent keeps this optimization infrastructure unchanged: it uses HRPO for the proposer and GRPO for the solver, and changes only the reward design and, optionally, the source-document selector.

3 Method

The difficulty reward of Eq. (2) encourages the proposer whenever the solver is uncertain about a generated question, but it does not verify whether the proposer’s answer is supported by any source span: the same reward is paid whether the underlying evidence is genuine or unrelated. Section 4.2 documents this failure mode empirically; we report it as a motivating diagnostic and devote the remainder of this section to the components that close the gap. Section 3.1 introduces the evidence verifier, which scores the proposer-emitted span by its causal effect on the solver’s answer accuracy. Section 3.2 reuses the same span as the supervision target during solver training. Section 3.3 describes an optional cluster bandit that diversifies the source-document distribution, and Section 3.4 explains how the proposer and solver are updated in two sequential phases. Figure 2 summarizes the resulting Phase A dataflow. The backbone, retriever, search tool, and policy-optimization framework remain unchanged.

3.1 Evidence verifier for the proposer

Proposer output. EVE-Agent changes the proposer output from a question–answer pair to a question–answer–evidence triple. Given a source document d and access to the search engine $\mathcal{R}$, the proposer generates

$$(q, a, e) \sim \pi_t^{\text{pro}}(\cdot \mid d, \mathcal{R}), \quad (8)$$

where q is the generated question, a is the target answer, and e is the evidence span. The evidence span must be copied verbatim from either the source document d or one of the snippets returned by the search engine during the proposer’s rollout. This constraint ensures that the evidence is not merely a free-form explanation, but a concrete text span that can be checked against the corpus.

After generation, we parse the output and apply a simple validity filter. Let $\text{NORM}(\cdot)$ denote the standard answer-normalization function that lowercases text, removes articles, strips punctuation, and collapses whitespace. A rollout is called *valid* if both q and a are non-empty and the normalized answer is not literally contained in the normalized question: $q \neq \emptyset$, $a \neq \emptyset$, $\text{NORM}(a) \not\subseteq \text{NORM}(q)$. The last condition removes degenerate questions that reveal their own answers. Invalid rollouts receive only the format-related reward defined below and are excluded from the evidence verifier.

Format reward. Before scoring whether the evidence helps the solver, we first assign a lightweight format reward that checks whether the proposer output is structurally usable. The reward $F_{\text{fmt}}(q, a, e, d, h) \in [0, 1]$ depends on the generated question q , answer a , evidence span e , source document d , and prescribed hop count h . Here, h denotes the intended number of reasoning steps, or equivalently the intended amount of multi-hop search behavior, for the generated question.

The format reward combines four equally weighted components. The first component is an integrity score that is always equal to 1 for parsed outputs that reach this stage. The remaining three components are sub-scores in $[0, 1]$: F_{think} rewards the presence of an explicit planning step, $F_{\text{tool}}(h)$ rewards the expected number and syntactic correctness of tool calls for an h -hop rollout, and F_{ans} rewards a concise answer that is consistent with the available context. We define

$$F_{\text{fmt}}(q, a, e, d, h) = \frac{1}{4} (1 + F_{\text{think}} + F_{\text{tool}}(h) + F_{\text{ans}}). \quad (9)$$

This term is intentionally simple: it ensures that the proposer follows the required output protocol, but it is not meant to judge whether the evidence actually supports the answer. That role is handled by the evidence verifier below. The precise definitions of the three sub-scores are provided in Appendix D.

Evidence-quality score. We now define the signal that measures whether the proposer-provided evidence is actually useful for answering the generated question. Consider a valid triple (q, a, e) , where q is the question, a is the target answer, and e is the evidence span emitted by the proposer. The key idea is to compare two answer probabilities: one when the solver is given both the question and the evidence, and one when it is given the question alone.

Let $\tilde{\pi}_{\text{sol},t}(\cdot \mid q, e)$ denote the current solver at round t under a single-turn answering protocol: the solver receives the question q and evidence span e , but it is not allowed to make any additional search calls. Similarly, let $\tilde{\pi}_{\text{aux},t}(\cdot \mid q)$ denote an auxiliary scorer under the same single-turn, search-disabled protocol, but conditioned only on the question q . We define

$$\begin{aligned} p_+^t(q, e, a) &:= \mathbb{P}_{\hat{a} \sim \tilde{\pi}_{\text{sol},t}(\cdot \mid q, e)} [\hat{a} = a], \\ p_-^t(q, a) &:= \mathbb{P}_{\hat{a} \sim \tilde{\pi}_{\text{aux},t}(\cdot \mid q)} [\hat{a} = a]. \end{aligned} \quad (10)$$

The first quantity, $p_+^t(q, e, a)$, is the probability of producing the correct answer when the evidence is provided. The second quantity, $p_-^t(q, a)$, is the probability of producing the same answer without access to that evidence. We define the evidence verifier as their difference:

$$V_t(q, e, a) := p_+^t(q, e, a) - p_-^t(q, a). \quad (11)$$

Since both probabilities lie in $[0, 1]$, the verifier score satisfies $V_t(q, e, a) \in [-1, 1]$.

This construction isolates the marginal contribution of the provided evidence. Search is disabled in both conditions, so the score does not reward the solver for finding new information after receiving the question. Instead, it asks a narrower and more auditable question: does the span e itself make the answer a easier to produce? A positive verifier score means that conditioning on e increases

the solver’s probability of generating the target answer. A score near zero means that the evidence provides little additional information beyond the question. A negative score indicates that the evidence makes the target answer less likely, which is consistent with the span being misleading or irrelevant.

In our experiments, the auxiliary scorer uses the same weights as the current solver. Thus, $\tilde{\pi}_{\text{aux},t}$ and $\tilde{\pi}_{\text{sol},t}$ differ only in their inputs: the former receives q , whereas the latter receives (q, e) . Under this choice, $V_t(q, e, a)$ directly measures the conditional answer-accuracy gain induced by the evidence for the current solver. The framework also supports a variant in which the auxiliary scorer is a separately hosted frozen model, but we do not use that configuration in the experiments reported here.

Estimator. The verifier score in Eq. (11) depends on two answer probabilities, which we estimate by Monte Carlo sampling. For a fixed valid triple (q, a, e) and an integer $m \geq 1$, we draw m independent answers from the solver with evidence,

$$\hat{a}_+^{(j)} \sim \tilde{\pi}_{\text{sol},t}(\cdot \mid q, e), \quad j = 1, \dots, m, \quad (12)$$

and m independent answers from the auxiliary scorer without evidence,

$$\hat{a}_-^{(j)} \sim \tilde{\pi}_{\text{aux},t}(\cdot \mid q), \quad j = 1, \dots, m. \quad (13)$$

We then estimate the evidence verifier by the difference between the two empirical accuracies:

$$\hat{V}_{t,m}(q, e, a) := \frac{1}{m} \sum_{j=1}^m \mathbf{1}\{\hat{a}_+^{(j)} = a\} - \frac{1}{m} \sum_{j=1}^m \mathbf{1}\{\hat{a}_-^{(j)} = a\}. \quad (14)$$

This estimator is unbiased, and its conditional variance is bounded by $1/(2m)$; both facts are formalized and proved as Proposition B.2 in Appendix B. It requires $2m$ additional single-turn decodes per valid proposer rollout— m with evidence and m without—an overhead that is modest because verifier rollouts are search-disabled, whereas the main solver rollouts are multi-turn interactions that may call the search engine several times. We use $m = 5$ throughout. A teacher-forced log-probability variant of the verifier is implemented but not used in the reported experiments because the sampling-based estimator is already sufficiently efficient.

Brevity bonus. The evidence span should be informative but not unnecessarily long. If the proposer copies a very long passage, the verifier becomes less useful because the span may contain many irrelevant facts or reveal the answer without identifying the specific supporting context. Conversely, an extremely short span may collapse to answer leakage rather than evidence. To encourage concise and targeted evidence, we introduce a brevity bonus. Let $|e|$ denote the number of tokens in the evidence span under the proposer tokenizer. We define

$$B(e) := \max\left(0, 1 - \frac{|e|}{L_{\max}}\right), \quad L_{\max} = 256. \quad (15)$$

The bonus is largest for short spans and decreases linearly with length. It becomes zero once the evidence span reaches $L_{\max}$ tokens. Thus, this term discourages copy-everything behavior while still allowing enough context for multi-hop or entity-rich questions.

Proposer reward. The final proposer reward combines structural validity, question difficulty, evidence usefulness, and evidence brevity. For a rollout with source document d , prescribed hop count h , generated triple (q, a, e) , and k correct solver predictions among n trials, we define

$$R_t^{\text{pro}}(q, e, a; d, h, k) = \frac{1}{2} F_{\text{fmt}}(q, a, e, d, h) + R_t^{\text{DZ}}(q, a; k) + \lambda_V V_t(q, e, a) + \lambda_B B(e), \quad (16)$$

where $\lambda_V \geq 0$ controls the strength of the evidence-verifiability term and $\lambda_B \geq 0$ controls the strength of the brevity bonus. In all experiments, we set

$$(\lambda_V, \lambda_B) = (0.5, 0.1). \quad (17)$$

Each term has a distinct role. The format reward ensures that the proposer follows the required output protocol. The difficulty reward encourages questions near the solver’s learning frontier. The verifier reward favors evidence spans that causally improve the solver’s ability to produce the target

answer. The brevity bonus discourages overly long evidence spans that would make the verifier less diagnostic. The prior self-evolving search agent [Yue et al., 2026] is recovered by setting

$$(\lambda_V, \lambda_B) = (0, 0), \quad (18)$$

while keeping the same difficulty and format components. We optimize the proposer with the HRPO objective in Eq. (6), replacing the original difficulty-only reward with R_t^{pro} and making no other changes to the optimization procedure.

3.2 Solver reward

After training the proposer with the reward in Eq. (16), we freeze it and use it to construct the solver-training data. Specifically, we roll out the trained proposer over the corpus. Each valid rollout produces a triple (q, a, e) , where q is the generated question, a is the target answer, and e is the proposer-provided evidence span. We then treat e as the gold evidence for training the solver. This choice is important: the same evidence span that the proposer was rewarded for producing is reused as the supervision target for the solver.

The solver is trained with the GRPO objective in Eq. (7). For each generated training instance, the solver is required to output both an answer $\hat{a}$ and an evidence span $\hat{e}$. We define the solver reward as

$$R^{\text{sol}}(\hat{a}, \hat{e}; a, e) = R_{\text{correct}}(\hat{a}, a) + \lambda_E R_{\text{evidence}}(\hat{e}, e), \quad (19)$$

where R_{correct} evaluates answer correctness and R_{evidence} evaluates evidence recovery. The answer reward is an exact-match indicator after standard answer normalization:

$$R_{\text{correct}}(\hat{a}, a) = \mathbf{1}\{\text{EM}(\hat{a}, a)\}. \quad (20)$$

Here, $\text{EM}(\hat{a}, a)$ equals true when the normalized predicted answer $\hat{a}$ exactly matches the normalized target answer a .

The evidence reward measures how well the solver’s extracted evidence span $\hat{e}$ matches the proposer-provided evidence span e . We use the SQuAD-style token-level F1 score:

$$R_{\text{evidence}}(\hat{e}, e) = \text{F1}_{\text{tok}}(\text{NORM}(\hat{e}), \text{NORM}(e)), \quad (21)$$

where NORM is the normalization function defined earlier and F1_{tok} is the harmonic mean of token-level precision and recall between the normalized predicted and target evidence spans. We set $\lambda_E = 0.3$ in all experiments. This reward encourages the solver not only to answer correctly, but also to recover the evidence that supports the answer.

3.3 Optional corpus selector

The default self-evolution loop samples source documents uniformly from the corpus. While simple, uniform sampling can concentrate training on a narrow set of documents or question patterns that happen to receive high reward early in training. This may reduce curriculum diversity: the proposer can repeatedly generate similar questions that lie near the solver’s current learning frontier, while underusing other topics and reasoning types.

To mitigate this issue, we include an optional corpus selector that can replace uniform document sampling. The selector is not required for the core evidence-verification mechanism, but it provides a way to diversify the self-generated curriculum. Its goal is to balance two forms of diversity. The first is topic diversity, controlled by clustering documents in embedding space. The second is question-type diversity, controlled by a small set of open-domain QA categories.

Concretely, a frozen sentence encoder maps each document $d \in \mathcal{D}$ to a fixed vector representation. At training round t , these document embeddings are partitioned into K_t clusters, where

$$K_t = K_0 + \lfloor \alpha t \rfloor. \quad (22)$$

Here, K_0 is the initial number of clusters, $\alpha \geq 0$ controls how quickly the clustering granularity increases, and $\lfloor \cdot \rfloor$ denotes the floor function. Larger values of K_t correspond to a finer partition of the corpus.

The selector uses two UCB1 bandits [Auer et al., 2002]. The first is a cluster bandit whose arms correspond to the K_t document clusters. The second is a task-type bandit whose arms correspond

to five question types: FACTUAL, COMPARISON, CAUSAL, TEMPORAL, and AGGREGATION. At each sampling step, the cluster bandit selects a topic cluster and the task-type bandit selects a desired question type. A source document is then sampled from the selected cluster with probability proportional to two factors: its distance from the cluster centroid and the inverse of its previous usage count. This favors documents that are both less typical within the cluster and less frequently sampled, encouraging coverage of boundary cases and reducing repeated use of the same documents.

The bandits are updated offline between self-evolution iterations. Their feedback reward combines a diversity term, which penalizes repeatedly selecting already overused clusters or task types, with a utility term, which compares the reward of the current generated sample to the running average reward for the corresponding arm. In this way, the selector encourages exploration without ignoring which parts of the corpus are currently useful for training. Since the selector is orthogonal to the evidence verifier, we do not isolate its empirical effect in the present experiments. The full algorithmic specification is given in Appendix F.

3.4 Two-phase training schedule

We train the proposer and the solver in two sequential phases rather than updating them jointly. This design has two motivations. First, it reduces compute: only one model is optimized with policy gradients at a time. Second, it improves stability. The evidence verifier depends on the solver, so updating the solver while simultaneously using it to score proposer outputs would make the reward landscape non-stationary and difficult to interpret.

In *Phase A*, the solver is kept fixed at its initialization, and the proposer is trained with HRPO using the reward R_t^{pro} in Eq. (16). The auxiliary scorer used in the verifier shares weights with this fixed solver. For each valid proposer rollout, the verifier score is estimated by the Monte Carlo estimator $\hat{V}_{t,m}$ in Eq. (14); we use $m = 5$ samples. Thus, Phase A teaches the proposer to generate not only difficult questions, but also evidence spans that improve the fixed solver’s ability to recover the target answer.

At the end of Phase A, we freeze the trained proposer and use it to generate the solver-training set. We roll it out over a held-out shard of the corpus with the full multi-turn search tool enabled. Each valid rollout contributes one triple (q, a, e) , where q is the generated question, a is the target answer, and e is the proposer-provided evidence span. We store e as the gold evidence for the corresponding question-answer pair.

In *Phase B*, the proposer is frozen, and the solver is trained with GRPO using the reward R^{sol} in Eq. (19). The solver is required to output both an answer and an evidence span, and it is rewarded for both answer correctness and evidence recovery. This phase transfers the evidence-verifiable curriculum produced by the proposer into the solver.

Relative to a standard self-evolving search agent, the modification is intentionally local. EVE-Agent keeps the backbone model, retriever, search tool, hop grouping, and the HRPO and GRPO optimization objectives unchanged. The core change is the reward design: the proposer is rewarded for generating useful evidence, and the solver is rewarded for recovering that evidence. Two basic guarantees of the reward design—a closed form and a unique interior maximizer for the inherited difficulty reward, and an unbiased, bounded-variance interpretation of the evidence verifier as a marginal answer-accuracy gain—are stated and proved in Appendix B, and Algorithms 1–2 in Appendix E give the full data flow.

4 Experiments

Our experimental study is organized around two questions, addressed in turn. First, does the difficulty-only reward of prior self-evolving search agents leave a measurable evidence-grounding gap? Section 4.2 answers this question and isolates the empirical motivation for the verifier. Second, does the evidence verifier of Eq. (11) close this gap while preserving—or improving—answer accuracy across benchmarks? Section 4.3 answers this question and isolates the verifier’s contribution under matched compute and matched search tools. Throughout the section we report per-benchmark numbers in tables and discuss the qualitative picture in prose; full quantitative details and additional protocol-specific information are deferred to Appendix C and Appendix H.

4.1 Experimental setup

Models and search tool. The proposer, the solver, and the auxiliary scorer in Eq. (11) share the Qwen2.5-3B-Instruct [Yang et al., 2024] backbone, and the auxiliary scorer reuses the current solver weights. All systems share the same retrieval pipeline: passages from the FlashRAG Wikipedia-2018 snapshot are encoded by E5-base-v2 and indexed by FAISS-IVF; each search call returns the top three passages, and a multi-turn rollout permits at most five assistant turns. Full indexing parameters and decoding settings are listed in Appendix C.

Training schedule and key hyperparameters. Both phases use a single $8\times B200$ node, a global batch size of 256, and run for 50 policy-gradient steps. The proposer-side coefficients (λ_V, λ_B) that enter the proposer reward of Eq. (16) are set to $(0.5, 0.1)$, and the brevity cutoff is $L_{\max} = 256$ tokens; the verifier Monte Carlo budget in Eq. (14) is $m = 5$. The solver-side coefficient λ_E in Eq. (19) is set to 0.3. Hop counts $h \in \{1, 2, 3, 4\}$ are sampled in ratio 4:3:2:1. Because the auxiliary scorer shares its weights with the solver, the verifier adds only $2m = 10$ single-turn, search-disabled decodes per valid proposer rollout, which is negligible compared with the multi-turn search rollouts. The full list of optimizer settings and additional implementation choices is given in Appendix C, with hyperparameters summarized in Appendix G.

Benchmarks and metrics. We evaluate on seven open-domain QA datasets: NaturalQuestions [Kwiatkowski et al., 2019], TriviaQA [Joshi et al., 2017], PopQA [Mallen et al., 2023], HotpotQA [Yang et al., 2018], 2WikiMultiHopQA [Ho et al., 2020], MuSiQue [Trivedi et al., 2022], and Bamboogle [Press et al., 2023]. Each system is required to emit an answer and a supporting evidence span. We report three complementary metrics: answer exact match (EM) after answer normalization; an evidence score judged by GPT-4.1 from the question, the gold answer, and the emitted span; and the joint rate at which the answer is correct *and* the evidence is judged supporting. The average column is the unweighted mean over the seven benchmarks.

Compared systems. We compare four systems trained or evaluated under matched protocols. *Initial (no search)* and *Initial (search)* are the untrained backbone evaluated without and with the search tool, respectively. *Dr. Zero* is a faithful re-implementation of Yue et al. [2026] under the same backbone, retrieval corpus, tool, rollout budget, and wall-clock budget. *EVE-Agent* adds the evidence-verifier reward and the matching solver evidence term while leaving the proposer, solver, backbone, and search tool otherwise identical to Dr. Zero. This design isolates the contribution of evidence-aware reward shaping.

4.2 Evidence-grounding bottleneck of prior systems

Objective. We first quantify the failure mode that motivates EVE-Agent: even when a self-evolving search agent attains competitive answer accuracy, the spans it emits as evidence may bear no real relation to the answer. The goal of this experiment is to verify that the difficulty-only reward of prior systems leaves a measurable gap between *producing* an answer and *justifying* it.

Protocol-specific details. On top of the shared setup of Section 4.1, we sample 1,000 test instances per benchmark, except for Bamboogle, for which we use the full 125-instance split. Each instance is decoded once by every system with the search tool enabled, and the emitted evidence span is passed to the external judge. We focus on a representative subset of four datasets that spans both single-hop and multi-hop regimes; the full breakdown is given in Appendix H.

Results. Table 1 reports the judged evidence score and the joint answer-and-evidence rate on the four-dataset subset; the corresponding answer-accuracy numbers are given in Appendix H. Two patterns are immediate. First, the prior self-evolving search agent does not narrow the evidence-quality gap relative to the untrained backbone, even though its answer accuracy is markedly higher: on the open-domain datasets the judge accepts only a similar fraction of its spans as supporting. Second, the joint correctness rate, which credits an instance only when the answer is right *and* the evidence is supporting, is correspondingly low for the prior system—comparable to the untrained baselines on most benchmarks. EVE-Agent improves both quantities on every dataset in this subset except 2WikiMultiHopQA, where the evidence score is competitive but not the best.

Table 1: Evidence-grounding diagnostics on a representative subset of benchmarks (1,000 test instances each, except Bamboogle with 125). *Prior* is a faithful re-implementation of Yue et al. [2026]. All systems use the same backbone and search tool; the external judge sees only the model-emitted evidence span. The full breakdown is given in Appendix H.

Method	Evidence score (judge)				Answer-and-evidence		
	NQ	TriviaQA	HotpotQA	2Wiki	NQ	TriviaQA	HotpotQA
Initial (no search)	0.274	0.424	0.247	0.199	0.052	0.180	0.048
Initial (search)	0.374	0.373	0.199	0.173	0.024	0.093	0.023
Prior [Yue et al., 2026]	0.242	0.289	0.209	0.205	0.021	0.098	0.035
EVE-Agent	0.484	0.582	0.332	0.166	0.242	0.342	0.152

Table 2: Answer accuracy (exact match) on seven open-domain QA benchmarks.

Method	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Avg
Initial (no search)	0.068	0.219	0.081	0.057	0.034	0.015	0.200	0.096
Initial (search)	0.032	0.125	0.055	0.029	0.013	0.010	0.024	0.041
Dr. Zero	0.069	0.257	0.134	0.110	0.077	0.055	0.104	0.115
EVE-Agent	0.289	0.437	0.300	0.209	0.176	0.050	0.088	0.221

Discussion. The bottleneck is not that prior systems omit evidence: Appendix H shows that they emit a syntactically valid evidence block in over 90% of rollouts, comparable to the initial backbone. The bottleneck is that the emitted block frequently fails to justify the predicted answer, which is consistent with the difficulty reward’s design: it audits whether a question is challenging, not whether it is supported. The evidence verifier of Section 3.1 addresses this by rewarding spans that causally raise the solver’s probability of producing the target answer.

4.3 Main results across benchmarks

Objective. Having identified the bottleneck, we now ask whether the evidence verifier closes it across the full benchmark suite without sacrificing answer accuracy. We compare EVE-Agent against the matched Dr. Zero re-implementation and the two untrained baselines on all seven datasets and report each of the three metrics in turn (answer EM, evidence score, joint correctness).

Protocol-specific details. We reuse the setup of Section 4.1 verbatim; the only change relative to Section 4.2 is that all seven benchmarks are now included. Decoding is greedy and the search tool is enabled for both the prior system and EVE-Agent. The external judge configuration is unchanged.

Answer accuracy. Answer exact match is the standard summary metric for open-domain QA; we report it first because a verifier that improved evidence quality at the cost of answer correctness would not be useful. Table 2 reports answer EM. EVE-Agent attains the best average EM and is strongest on five of the seven benchmarks—NQ, TriviaQA, PopQA, HotpotQA, and 2WikiMultiHopQA—each of which has a meaningfully large evaluation pool. On MuSiQue and Bamboogle the picture is mixed: Dr. Zero edges out EVE-Agent on MuSiQue by a small margin, and the untrained no-search backbone is highest on the small Bamboogle split. Overall, the evidence-oriented reward does not trade off answer correctness for explanation format; on the contrary, accuracy improves under the same compute and search budget as the prior self-evolving search agent.

Evidence quality. Evidence quality is the central diagnostic of the bottleneck of Section 4.2: only this metric reveals whether the predicted answer is paired with a source span that a careful reader could use to verify it. Table 3 reports the judged evidence score across all seven benchmarks. EVE-Agent substantially improves evidence support on all single-hop benchmarks—NQ, TriviaQA, and PopQA—and on HotpotQA, and the average evidence score across the seven benchmarks is the highest of the four systems by a clear margin. The remaining benchmarks are mixed: on 2WikiMultiHopQA Dr. Zero is slightly stronger; on MuSiQue the search-equipped initial backbone narrowly edges out EVE-Agent; and on Bamboogle the small evaluation split favors the no-search backbone. These exceptions notwithstanding, the verifier-shaped reward produces evidence that the external judge accepts as supporting more often than any baseline, including the search-equipped initial backbone.

Table 3: Evidence score judged by an external GPT-4.1 evaluator. The judge sees only the question, the gold answer, and the model-emitted evidence span.

Method	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Avg
Initial (no search)	0.274	0.424	0.153	0.247	0.199	0.093	0.376	0.252
Initial (search)	0.374	0.373	0.298	0.199	0.173	0.103	0.176	0.242
Dr. Zero	0.242	0.289	0.208	0.209	0.205	0.086	0.128	0.195
EVE-Agent	0.484	0.582	0.392	0.332	0.166	0.101	0.136	0.313

Table 4: Joint answer-and-evidence score. An instance is counted only when the answer is correct *and* the emitted evidence is judged supporting by the external judge.

Method	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Avg
Initial (no search)	0.052	0.180	0.069	0.048	0.016	0.015	0.184	0.081
Initial (search)	0.024	0.093	0.042	0.023	0.008	0.008	0.024	0.032
Dr. Zero	0.021	0.098	0.059	0.035	0.035	0.023	0.040	0.044
EVE-Agent	0.242	0.342	0.264	0.152	0.070	0.038	0.064	0.167

Crucially, this improvement is obtained with the same backbone, retriever, and search tool used by Dr. Zero, so it can be attributed to the reward design rather than to additional capacity or retrieval signal.

Joint answer-and-evidence correctness. The joint metric is the strictest of the three: an instance is credited only when the predicted answer matches the gold answer *and* the emitted span is judged supporting. From the perspective of evidence verifiability, this is the regime that matters most, because it isolates the cases in which the agent’s output is simultaneously correct and auditable. Table 4 reports the result across all seven benchmarks. EVE-Agent obtains the best average score and is strongest on six of seven benchmarks; on the average column it improves over the matched Dr. Zero re-implementation by a wide margin. The improvement is not a redistribution between the previous two metrics: EVE-Agent’s answer-accuracy gains are concentrated in instances whose evidence is also judged sufficient, which is precisely the population that can be reused as reliable training signal for downstream learning or as a basis for human inspection of the curriculum.

Discussion. Read together, Tables 2–4 support the central claim of the paper. The evidence verifier of Eq. (11) produces a training-time signal whose gains are not bought by relaxing answer correctness, by overfitting to evidence presence, or by exploiting additional compute relative to the prior system. The answer accuracy and the evidence quality improve together, and the strict joint metric—the metric most aligned with evidence verifiability—exhibits the largest relative gap to prior work. The pattern is consistent across single-hop and multi-hop benchmarks, with the two exceptions involving either a small evaluation pool (Bamboogle, 125 instances) or a regime in which the prior system already matches the untrained backbone on evidence grounding (2WikiMultiHopQA). The persistence of the improvement on the strict joint metric is, in our view, the cleanest summary of the verifier’s effect: under matched compute and matched search tools, EVE-Agent generates curricula and trains solvers whose answers are simultaneously more often correct and more often verifiable.

5 Related work

Data-free self-evolving reasoning and search agents. Absolute Zero [Zhao et al., 2025] introduced the data-free paradigm with a Python-interpreter oracle and R-Zero [Huang et al., 2026] generalized it through a challenger–solver decoupling. Recent work [Yue et al., 2026] ports the loop to multi-turn search agents with hop-grouped relative policy optimization; SAGE [Peng et al., 2026] adds multi-agent critics and AReaL-SEA [Gao et al., 2026] adds multi-turn tool use to the same template. EVE-Agent differs in injecting a data-free *evidence verifier* so that the proposer’s reward depends on whether the emitted span causally helps the trained solver, not only on whether the solver is uncertain.

Verifier-based and retrieval-augmented RL. Verifier-based rewards [Lambert et al., 2024, Shao et al., 2024, Cobbe et al., 2021] and knowledge-graph verifiers [Yuan et al., 2026] either assume an external oracle or pay a heavy graph-construction cost; in contrast, the verifier of Eq. (11) is defined

entirely through the trained solver, the proposer-emitted evidence, and the corpus. Search-R1 [Jin et al., 2025] and R1-Searcher [Song et al., 2025] are retrieval-augmented RL agents trained on supervised question–answer pairs, and Self-RAG and IRCoT provide self-critic and interleaved retrieval templates [Asai et al., 2024, Trivedi et al., 2023] that we inherit at the proposer level.

Curriculum diversity. Semantic diversity rewards [Wan et al., 2026] and the R-Diverse template of Li et al. [2026] act *after* sampling by down-weighting near-duplicates. The optional selector of Section 3.3 instead acts *before* sampling via a cluster bandit, in the lineage of UCB1 [Auer et al., 2002] and non-stationary bandit analyses [Garivier and Moulines, 2011, Lattimore and Szepesvári, 2020]; curriculum-learning methods [Graves et al., 2017, Matiisen et al., 2020] provide background on schedule design more broadly.

Evidence-grounded evaluation of search agents. Evidence-aware evaluation benchmarks [Glockner et al., 2025] formalize the question of whether a model emits a supporting span; our verifier turns this diagnostic into a training-time signal.

6 Conclusion

We argued that *evidence verifiability* should be treated as a first-class property of data-free self-evolving search agents. In existing systems, the proposer is rewarded for generating difficult questions, but no reward checks whether the predicted answer is grounded in a span that a careful reader could use to verify it. We documented the resulting bottleneck empirically on representative open-domain QA benchmarks: a faithful re-implementation of the prior self-evolving search agent answers questions competitively yet emits evidence whose judged quality is no better than that of an untrained backbone.

EVE-Agent closes this gap with a minimal extension that is local to the reward. The proposer is required to emit a verbatim source span together with its question and answer, and the span is credited only when its inclusion causally improves the current solver’s ability to recover the target answer; the solver is in turn trained to reproduce both the answer and the supporting span. The backbone, the retriever, the search tool, and the policy-optimization framework remain unchanged.

Across seven open-domain QA benchmarks, under matched compute and matched search tools, this single change yields curricula and solvers whose outputs are simultaneously more often correct and more often verifiable: EVE-Agent improves on answer accuracy, on judged evidence quality, and—most importantly—on the strict joint metric that credits an instance only when both are adequate. The resulting training loop is auditable by construction. Every generated example carries an inspectable source span whose contribution can be checked against the current solver, and the curriculum can be reviewed instance by instance rather than treated as an opaque collection of question–answer pairs.

We see this as a small but concrete step toward *safer self-evolution*. As data-free agents increasingly write their own training data, evidence grounding becomes a prerequisite for trust: an agent’s improvements should not rely on training signal that cannot itself be verified, and the soundness of the learning process should be checkable from the data the agent produces, not only from the metrics it eventually reports. EVE-Agent treats evidence as a training-time object that can be inspected, scored, and reused; we expect that incorporating similar verifiability constraints will become standard practice as self-evolving search agents are deployed in settings where their training data, and not only their final answers, must be trusted.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Jiaxuan Gao, Jiaao Chen, Chuyi He, Shusheng Xu, Di Jin, and Yi Wu. From self-evolving synthetic data to verifiable-reward RL: Post-training multi-turn interactive tool-using agents. *arXiv preprint arXiv:2601.22607*, 2026.
- Aurélien Garivier and Eric Moulines. On upper-confidence bound policies for switching bandit problems. In *Algorithmic Learning Theory (ALT)*, 2011.
- Max Glockner, Xiang Jiang, Leonardo F. R. Ribeiro, Iryna Gurevych, and Markus Dreyer. NeoQA: Evidence-based question answering with generated news events. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *International Conference on Machine Learning (ICML)*, 2017.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *International Conference on Computational Linguistics (COLING)*, 2020.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2508.05004.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.
- Tom Kwiatkowski et al. Natural questions: A benchmark for question answering research. In *Transactions of the Association for Computational Linguistics (TACL)*, 2019.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester J Vedelgo Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Gengsheng Li, Jinghan He, Shijie Wang, Dan Zhang, Ruiqi Liu, Renrui Zhang, Zijun Yao, Junfeng Fang, Haiyun Guo, and Jinqiao Wang. R-Diverse: Mitigating diversity illusion in self-play LLM training. *arXiv preprint arXiv:2602.13103*, 2026.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Association for Computational Linguistics (ACL)*, 2023.

- Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher–student curriculum learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3732–3740, 2020.
- Yulin Peng, Xinxin Zhu, Chenxing Wei, Nianbo Zeng, Leilei Wang, Ying Tiffany He, and F. Richard Yu. SAGE: Multi-agent self-evolution for LLM reasoning. *arXiv preprint arXiv:2603.15255*, 2026.
- Ofir Press et al. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2302.04761.
- Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-Searcher: Incentivizing the search capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- Harsh Trivedi et al. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics (TACL)*, 2022.
- Zhongwei Wan, Yun Shen, Zhihao Dou, Donghao Zhou, Yu Zhang, Xin Wang, Hui Shen, Jing Xiong, Chaofan Tao, Zixuan Zhong, Peizhou Huang, and Mi Zhang. DSDR: Dual-scale diversity regularization for exploration in LLM reasoning. *arXiv preprint arXiv:2602.19895*, 2026.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- Zhonghang Yuan, Zhefan Wang, Fang Hu, Zihong Chen, Huanjun Kong, Songyang Zhang, Wanli Ouyang, and Nanqing Dong. Knowledge-to-verification: Unlocking reinforcement learning with verifiable rewards for LLMs in knowledge-intensive domains. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.
- Zhenrui Yue, Kartikeya Upasani, Xianjun Yang, Suyu Ge, Shaoliang Nie, Yuning Mao, Zhe Liu, and Dong Wang. Dr. Zero: Self-evolving search agents without training data. *arXiv preprint arXiv:2601.07055*, 2026.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

A Notation

Table 5 collects the principal symbols used in the main text. Throughout the appendix, log-probabilities are natural logarithms, indicator functions take values in $\{0, 1\}$, and KL divergences are non-negative and can be infinite.

Table 5: Glossary of the principal symbols used throughout the paper.

Symbol	Meaning	First used
$\mathcal{D}$	source corpus	Section 2
$d \in \mathcal{D}$	source document	Section 2
q, a, e	question, answer, evidence span (e verbatim from d or a snippet)	Section 2
$h \in \{1, 2, 3, 4\}$	prescribed hop count of the rollout	Section 3.1
$\pi_t^{\text{pro}}, \pi_t^{\text{sol}}$	proposer / solver policies at round t	Section 2
$\tilde{\pi}_{\text{sol}, t}$	solver under the single-turn search-disabled prompt	Eq. (10)
$\tilde{\pi}_{\text{aux}, t}$	auxiliary scorer (in our runs: weight-swapped solver)	Eq. (10)
$p_+^t(q, e, a), p_-^t(q, a)$	with-evidence and no-evidence answer accuracies	Eq. (10)
$V_t(q, e, a)$	evidence-quality score $p_+^t - p_-^t$	Eq. (11)
$\hat{V}_{t, m}$	Monte Carlo estimator of V_t with m samples	Eq. (14)
$B(e)$	brevity bonus, $L_{\max} = 256$	Eq. (15)
F_{fmt}	4-component proposer format reward	Eq. (9)
$R_t^{\text{DZ}}(q, a; k)$	Dr. Zero difficulty reward, $k \sim \text{Bin}(n, p_t^{\text{sol}})$	Eq. (2)
$\phi_n(p)$	population mean of R_t^{DZ}	Eq. (4)
$R_t^{\text{pro}}, R_t^{\text{sol}}$	proposer (Phase A) and solver (Phase B) rewards	Eqs. (16), (19)
$R_{\text{correct}}, R_{\text{evidence}}$	solver answer EM and evidence-F1 sub-rewards	Eq. (19)
$\lambda_V, \lambda_B, \lambda_E$	EVE-Agent reward coefficients	Eqs. (16), (19)
$z_d = \phi(d) \in \mathbb{R}^D$	document embedding (E5-base-v2)	Section 3.3
$K_t = K_0 + \lfloor \alpha t \rfloor$	cluster count schedule	Section 3.3
$\mathcal{C}_t = \{C_{t, k}\}_{k=1}^{K_t}$	active clusters at round t	Section 3.3
$N_k, \bar{R}_k$	UCB1 pull count and empirical mean for arm k	Appendix F
$\beta, \lambda_u, \varepsilon$	UCB exploration weight, utility weight, log smoothing	Appendix F

B Theoretical results

This appendix collects two basic properties of the reward design used throughout the paper. Lemma B.1 characterizes the inherited difficulty reward as a unimodal function of the solver’s per-trial success probability, and Proposition B.2 formalizes the evidence verifier as a marginal answer-accuracy gain with an unbiased, bounded-variance Monte Carlo estimator. Both results are referenced from the main text but were moved here to keep the body focused on the empirical contribution.

B.1 Closed form and saturation of the difficulty reward

Lemma B.1 (Closed form and saturation of the difficulty reward). *Let $n \geq 2$ be the number of independent solver predictions, and let $p \in [0, 1]$ be the probability that a single solver prediction matches the target answer. If $k \sim \text{Bin}(n, p)$ denotes the number of correct predictions among the n trials, then*

$$\mathbb{E}_{k \sim \text{Bin}(n, p)} [R_t^{\text{DZ}}(q, a; k)] = \phi_n(p), \quad (23)$$

where

$$\phi_n(p) := \frac{n}{n-1} (1-p) (1 - (1-p)^{n-1}). \quad (24)$$

The function $\phi_n : [0, 1] \rightarrow [0, 1]$ is continuous and unimodal, with $\phi_n(0) = \phi_n(1) = 0$ and a unique interior maximizer

$$p^* = 1 - n^{-1/(n-1)}. \quad (25)$$

Discussion. Lemma B.1 makes explicit what the difficulty reward in Eq. (2) encourages. The expected reward vanishes both when the solver almost never answers correctly and when it almost always does, and is largest at an intermediate success probability p^* . This frontier is defined entirely by answer accuracy, however, and does not address whether the generated answer is supported by any source span—the motivation for the evidence verifier of Eq. (11).

Proof of Lemma B.1. Write $p_t^{\text{sol}}(q, a) = p$ and let $k \sim \text{Bin}(n, p)$. From Eq. (2),

$$\begin{aligned}\mathbb{E}[R_t^{\text{DZ}}] &= \sum_{k=1}^{n-1} \binom{n}{k} p^k (1-p)^{n-k} \frac{n-k}{n-1} \\ &= \frac{1}{n-1} \left[\sum_{k=0}^n \binom{n}{k} p^k (1-p)^{n-k} (n-k) - n(1-p)^n - 0 \right],\end{aligned}$$

where the $k = 0$ term contributes $n(1-p)^n$ and the $k = n$ term contributes 0. The full sum is the binomial mean of $n - k$, which equals $n(1-p)$, so the bracket equals

$$n(1-p) - n(1-p)^n = n(1-p)(1 - (1-p)^{n-1}).$$

Dividing by $n-1$ gives $\phi_n(p) = \frac{n}{n-1}(1-p)(1 - (1-p)^{n-1})$.

Continuity is immediate from the polynomial form. The boundary values are $\phi_n(0) = 0$ and $\phi_n(1) = 0$ by the factor $(1-p)$. Differentiating yields $\phi'_n(p) = \frac{n}{n-1}[n(1-p)^{n-1} - 1]$, which vanishes exactly at $p^* = 1 - n^{-1/(n-1)} \in (0, 1)$. Since $\phi'_n(0) > 0$ and $\phi'_n(1) < 0$, this critical point is the unique interior maximum. Finally, $\phi_n(p) \in [0, 1]$ for every $p \in [0, 1]$ because $R_t^{\text{DZ}}(q, a; k) \in [0, 1]$ for every realization of k , so its expectation also lies in $[0, 1]$. $\square$

B.2 Evidence verifier as marginal answer-accuracy gain

Proposition B.2 (Evidence verifier as marginal answer-accuracy gain). *For any valid triple (q, a, e) and any training round t , the evidence verifier in Eq. (11) satisfies*

$$V_t(q, e, a) = \mathbb{E}_{\hat{a} \sim \tilde{\pi}_{\text{sol},t}(\cdot | q, e)} [\mathbf{1}\{\hat{a} = a\}] - \mathbb{E}_{\hat{a} \sim \tilde{\pi}_{\text{aux},t}(\cdot | q)} [\mathbf{1}\{\hat{a} = a\}], \quad (26)$$

and therefore $V_t(q, e, a) \in [-1, 1]$. When the auxiliary scorer shares its weights with the solver, the two distributions differ only in whether the evidence span e is provided; in that case $V_t(q, e, a)$ is exactly the increase in the solver’s answer probability induced by conditioning on e , and $V_t(q, e, a) = 0$ whenever $\tilde{\pi}_{\text{sol},t}(\cdot | q, e) = \tilde{\pi}_{\text{sol},t}(\cdot | q)$ for every e . Moreover, the Monte Carlo estimator $\hat{V}_{t,m}$ in Eq. (14) is unbiased,

$$\mathbb{E}[\hat{V}_{t,m}(q, e, a) | q, e, a] = V_t(q, e, a), \quad (27)$$

and its conditional variance satisfies

$$\text{Var}[\hat{V}_{t,m}(q, e, a) | q, e, a] \leq \frac{1}{2m}. \quad (28)$$

Discussion. Proposition B.2 clarifies the interpretation of the verifier introduced in Section 3.1. A positive $V_t(q, e, a)$ means that the proposed evidence span makes the target answer more likely for the current solver, a value near zero means that the solver could already infer the answer from the question alone, and a negative value indicates a misleading or irrelevant span. The verifier thus turns evidence grounding into a reward-level quantity that can be optimized without oracle labels. We do not claim that this reward induces a non-vanishing policy gradient for every proposer parameterization; such a claim would require additional assumptions on the policy class and the optimization dynamics.

Proof of Proposition B.2. The identity (26) is immediate from the definitions of V_t and the two pointwise accuracies in Eqs. (10)–(11):

$$V_t(q, e, a) = \tilde{\pi}_{\text{sol},t}(a | q, e) - \tilde{\pi}_{\text{aux},t}(a | q),$$

which is the difference between two probabilities and hence lies in $[-1, 1]$. In the weight-swap configuration $\tilde{\pi}_{\text{aux},t} \equiv \tilde{\pi}_{\text{sol},t}$, so $V_t(q, e, a) = \tilde{\pi}_{\text{sol},t}(a | q, e) - \tilde{\pi}_{\text{sol},t}(a | q)$. If $\tilde{\pi}_{\text{sol},t}(\cdot | q, e) = \tilde{\pi}_{\text{sol},t}(\cdot | q)$ for every e , then $V_t(q, e, a) = 0$ identically.

For the Monte Carlo estimator, each indicator $\mathbf{1}\{\hat{a}_+^{(j)} = a\}$ is a Bernoulli random variable with mean $\tilde{\pi}_{\text{sol},t}(a \mid q, e)$, and the analogous statement holds for $\mathbf{1}\{\hat{a}_-^{(j)} = a\}$. By linearity of expectation, $\mathbb{E}[\hat{V}_{t,m} \mid q, e, a] = V_t(q, e, a)$, establishing (27). The variance of a Bernoulli random variable with parameter p is $p(1-p) \leq 1/4$, so each of the two empirical means in Eq. (14) has conditional variance at most $1/(4m)$. Because the two empirical means are computed from independent samples by construction of the rollout schedule,

$$\text{Var}[\hat{V}_{t,m} \mid q, e, a] \leq \frac{1}{4m} + \frac{1}{4m} = \frac{1}{2m},$$

which proves (28). $\square$

C Additional implementation details

This appendix expands on the experimental setup of Section 4.1. The goal is to provide enough detail to reproduce all numbers in Tables 1–4; nothing in this appendix changes any reward equation in the main text.

Backbone and tokenization. All four compared systems use the Qwen2.5-3B-Instruct backbone [Yang et al., 2024], the same tokenizer, and the same chat template. The auxiliary scorer in Eq. (11) reuses the current solver weights; we refer to this implementation as the *weight-swap* configuration. The framework also supports a separately hosted frozen auxiliary scorer, but we do not use that configuration in any reported experiment.

Retrieval pipeline. The retrieval corpus is the FlashRAG Wikipedia-2018 snapshot ($\approx 21\text{M}$ passages). Passages are encoded with E5-base-v2 using mean pooling and unit-norm L^2 normalization, and indexed with FAISS-IVF over 4,096 centroids with $\text{nprobe} = 64$. The search tool returns the top-3 passages per query. Three CPU retrieval servers run in parallel to amortize tool latency.

Rollout protocol. A multi-turn proposer or solver rollout permits at most five assistant turns and at most 512 tokens per tool response. The single-turn protocol used by the evidence verifier of Section 3.1 disables the search tool and limits the assistant to one turn; this is the configuration in which p_+^t and p_-^t of Eq. (10) are computed.

Data preparation. Proposer training prompts are drawn from the FlashRAG NQ–HotpotQA mixture, rebound to hop counts $h \in \{1, 2, 3, 4\}$ in the ratio 4:3:2:1. To construct the Phase B training set we roll out the frozen Phase A proposer on the same corpus with five samples per prompt and retain only valid triples; the proposer-emitted span e becomes the gold evidence for the corresponding question–answer pair.

Optimization. Phase A runs 50 HRPO steps with global batch size 256, micro-batch 2 per GPU, learning rate $1\text{e-}6$, warmup ratio 0.03, gradient clipping at 0.1, the KL regularizer disabled, no nested grouping, and a verifier Monte Carlo budget $m = 5$. Phase B runs 50 GRPO steps with global batch size 256, micro-batch 8 per GPU, group size 5, and the same optimizer settings as Phase A; the evidence-F1 coefficient is $\lambda_E = 0.3$. Both phases use a single $8 \times \text{B200}$ node.

Inference for the evaluation tables. For Tables 1–4, each system is decoded greedily on each evaluation instance with the search tool enabled. The external judge for the evidence score is GPT-4.1, queried with the question, the gold answer, and the model-emitted span; the judge returns a binary decision over whether the question paired with the span is sufficient to recover the gold answer. The joint metric counts an instance as correct only when both the answer is exact-match equal to the gold answer (after normalization) and the judge labels the evidence as supporting.

D Format reward decomposition

The four sub-rewards of Eq. (9) are defined as follows. Given a parsed proposer output with h prescribed hops, T_{ass} assistant turns, T_{think} assistant turns that begin with a planning step, and T_{tc}

syntactically valid tool-call emissions whose number matches the number of returned tool responses,

$$F_{\text{think}} = \frac{T_{\text{think}}}{\max(1, T_{\text{ass}})},$$

$$F_{\text{tool}}(h) = \begin{cases} 1 & h = 1, \\ \min(\frac{1+T_{\text{tc}}}{h}, 1) & h > 1, T_{\text{tc}} = \text{\#returned responses}, \\ 0 & h > 1, \text{otherwise}, \end{cases}$$

and $F_{\text{ans}} \in \{0, \frac{1}{2}, 1\}$ rewards a short, in-context answer: writing $\tilde{a} := \text{NORM}(a)$ and letting Σ denote the concatenation of the source document with the returned tool responses,

$$F_{\text{ans}} = \begin{cases} 1 & \tilde{a} \in \{\text{yes}, \text{no}\} \text{ or } (\tilde{a} \subseteq \text{NORM}(\Sigma) \text{ and } |\tilde{a}|_{\text{words}} \leq 5), \\ \frac{1}{2} & \tilde{a} \subseteq \text{NORM}(\Sigma) \text{ and } 5 < |\tilde{a}|_{\text{words}} \leq 10, \\ 0 & \text{otherwise.} \end{cases}$$

Rollouts with empty q or empty a receive $F_{\text{fmt}} = 0$ and are filtered out before the verifier.

E Pseudocode

Algorithm 1 describes one Phase A HRPO iteration with the evidence verifier; Algorithm 2 describes one Phase B GRPO iteration with the solver reward of Eq. (19). In Algorithm 1, the auxiliary scorer shares its weights with the current solver and produces both the with-evidence and the no-evidence single-turn rollouts.

Algorithm 1 One Phase A iteration of EVE-Agent (proposer)

```

1: Input: proposer  $\pi_t^{\text{pro}}$ , solver  $\pi^{\text{sol}}$  (frozen), corpus  $\mathcal{D}$ , hop pmf, coefficients  $(\lambda_V, \lambda_B)$ , MC budget  $m$ , group size  $n$ 
2: Sample  $\{d_i\}_{i=1}^N \stackrel{\text{iid}}{\sim} \text{Unif}(\mathcal{D})$  and  $h_i \sim \text{hop pmf}$ 
3: for  $i = 1, \dots, N$  do
4:    $(q_i, a_i, e_i) \sim \pi_t^{\text{pro}}(\cdot \mid d_i, h_i, \mathcal{R})$ 
5:    $F_{\text{fmt},i} \leftarrow \text{COMPUTEFORMATSCORE}(q_i, a_i, e_i, d_i, h_i)$ 
6:   Mark  $i$  invalid if parse fails or  $F_{\text{fmt},i} = 0$ 
7: end for
8: for each valid  $i$  do
9:   Sample  $\{\hat{a}_j^i\}_{j=1}^n \sim \pi^{\text{sol}}(\cdot \mid q_i, \mathcal{R})$   $\triangleright$  multi-turn, with search
10:   $k_i \leftarrow \sum_{j=1}^n \mathbf{1}\{\hat{a}_j^i = a_i\}$ 
11:   $R_i^{\text{DZ}} \leftarrow \mathbf{1}\{0 < k_i < n\}(n - k_i)/(n - 1)$ 
12:  Sample  $\{\hat{a}_+^{(j),i}\}_{j=1}^m \sim \tilde{\pi}^{\text{sol}}(\cdot \mid q_i, e_i)$   $\triangleright$  single-turn, no search
13:  Sample  $\{\hat{a}_-^{(j),i}\}_{j=1}^m \sim \tilde{\pi}^{\text{aux}}(\cdot \mid q_i)$   $\triangleright$  shares weights with the solver
14:   $\hat{V}_i \leftarrow \frac{1}{m} \sum_j \mathbf{1}\{\hat{a}_+^{(j),i} = a_i\} - \frac{1}{m} \sum_j \mathbf{1}\{\hat{a}_-^{(j),i} = a_i\}$ 
15:   $B_i \leftarrow \max(0, 1 - |e_i|/L_{\text{max}})$ 
16:   $R_i^{\text{pro}} \leftarrow \frac{1}{2}F_{\text{fmt},i} + R_i^{\text{DZ}} + \lambda_V \hat{V}_i + \lambda_B B_i$ 
17: end for
18: for invalid  $i$  do
19:   $R_i^{\text{pro}} \leftarrow \frac{1}{2}F_{\text{fmt},i}$ 
20: end for
21: Compute hop-grouped advantages  $A_{i,h}$  by Eq. (5)
22: Update  $\pi^{\text{pro}}$  with one HRPO step on  $\{(q_i, a_i, e_i), A_{i,h}\}$ 

```

F Selector: full specification

This appendix gives the complete specification of the cluster-bandit selector introduced in Section 3.3.

Algorithm 2 One Phase B iteration of EVE-Agent (solver)

- 1: **Input:** Phase A proposer checkpoint (frozen), solver π_t^{sol} , training shard $\{(q_i, a_i, e_i)\}_{i=1}^N$ generated by the Phase A proposer, GRPO group size n , coefficient λ_E
 - 2: **for** $i = 1, \dots, N$ **do**
 - 3: Sample $\{(\hat{a}_j^i, \hat{e}_j^i)\}_{j=1}^n \sim \pi_t^{\text{sol}}(\cdot \mid q_i, \mathcal{R})$
 - 4: **for** $j = 1, \dots, n$ **do**
 - 5: $R_{\text{correct},j}^i \leftarrow \mathbf{1}\{\text{EM}(\hat{a}_j^i, a_i)\}$
 - 6: $R_{\text{evidence},j}^i \leftarrow \text{F1}_{\text{tok}}(\text{NORM}(\hat{e}_j^i), \text{NORM}(e_i))$
 - 7: $R_j^{\text{sol},i} \leftarrow R_{\text{correct},j}^i + \lambda_E R_{\text{evidence},j}^i$
 - 8: **end for**
 - 9: **end for**
 - 10: Update π^{sol} with one GRPO step of Eq. (7) on $\{R_j^{\text{sol},i}\}$
-

Document embeddings. A frozen sentence encoder ϕ (E5-base-v2 with mean pooling and L^2 normalization) maps each $d \in \mathcal{D}$ to $z_d = \phi(d) \in \mathbb{R}^D$, $D = 768$. Embeddings are precomputed once per corpus.

Adaptive clustering. At round t , the cluster set $\mathcal{C}_t = \{C_{t,k}\}_{k=1}^{K_t}$ is obtained by mini-batch k -means on $\{z_d\}_{d \in \mathcal{D}}$ with a granularity schedule

$$K_t = K_0 + \lfloor \alpha t \rfloor, \quad K_0 = 10, \alpha \geq 0. \quad (29)$$

With $\alpha = 0$ the clustering is fixed throughout training; with $\alpha > 0$, whenever $K_t > K_{t-1}$ we re-cluster the corpus and *inherit* the bandit statistics across the split. Concretely, each new centroid is assigned to its nearest old centroid by Euclidean distance, and if old cluster j has n_{children} new children with counts N_j and reward sum S_j , each child starts with $\max(\lfloor N_j/n_{\text{children}} \rfloor, 1)$ pulls and S_j/n_{children} reward. Parent statistics are split across children rather than discarded; in particular, we do not reset arm statistics on split.

UCB1 bandits. Two independent UCB1 bandits run side by side; both initialize every arm with one virtual pull and zero virtual reward so the exploration bonus is finite from round 1. For arm k with N_k pulls, reward sum S_k , and total pulls $N_{\text{tot}} = \sum_j N_j$, the UCB score is

$$U_k = \frac{S_k}{\max(N_k, 1)} + \beta \sqrt{\frac{\log \max(N_{\text{tot}}, 1)}{\max(N_k, 1)}}, \quad \beta > 0, \quad (30)$$

with $\beta = 1$ throughout. When a single arm is required, we use the deterministic $\arg \max_k U_k$; when a batch of $n > 1$ arms is required, the batch is drawn from a softmax over U_k rescaled by the empirical standard deviation of the U -values, which keeps the batch diverse without sacrificing the exploration bias of UCB1.

Cluster and task arm sets. The cluster bandit has K_t arms; arm k corresponds to the cluster $C_{t,k}$. The task-type bandit has five arms with labels FACTUAL, COMPARISON, CAUSAL, TEMPORAL, and AGGREGATION. The hop count is already controlled independently through the hop pmf and is therefore not a task type.

Within-cluster sampling. Let u_d^t be the number of times document d has been previously sampled at round t and let $\delta_{d,k} = \|z_d - \mu_k\|_2$ be its Euclidean distance to the cluster centroid μ_k . Inside the chosen cluster C_{t,k_t} , a document is drawn with probability

$$\mathbb{P}[d \mid k_t] \propto \delta_{d,k_t} \cdot \frac{1}{1 + u_d^t}, \quad d \in C_{t,k_t}. \quad (31)$$

The first factor biases sampling toward the cluster boundary, where atypical entities concentrate; the second is a soft sampling-without-replacement that discourages revisiting heavily-used documents inside the same cluster. If every score collapses to zero (e.g. for an empty cluster), the policy falls back to a uniform draw.

Between-iteration reward feedback. The selector is updated between self-evolution iterations rather than within a single policy-gradient step. After each iteration, we read (i) the generated solver-training samples, annotated with the selector’s chosen cluster id and task-type id per row, and (ii) the run-level summary score $R_{\text{run}} \in [0, 1]$ of the just-finished iteration (the average of the latest proposer and solver mean rewards). The per-sample reward is the product of R_{run} and a bounded quality proxy $Q_i \in [0, 1]$ defined for sample i as

$$Q_i = q_{\text{len}}(|q_i|) \cdot q_{\text{ans}}(|a_i|) \cdot \frac{1}{\sqrt{\max(D_i, 1)}}, \quad (32)$$

where $q_{\text{len}}(\ell) = 1$ if $20 \leq \ell \leq 220$ and 0.65 otherwise, $q_{\text{ans}}(\ell) = 1$ if $1 \leq \ell \leq 80$ and 0.55 otherwise, and $D_i \geq 1$ is the number of duplicates of the prompt string in the batch. Let $r_i := R_{\text{run}} Q_i$ be the resulting per-sample reward. Conditional on the cluster id $k(i)$, the selector reward fed back to the bandit is

$$R_i^{\text{sel}} = \underbrace{-\log(N_{k(i)}/N_{\text{tot}} + \varepsilon)}_{=: R_{\text{div}}(k(i))} + \lambda_u \underbrace{(r_i - \bar{r}_{k(i)}^{\text{prev}})}_{=: R_{\text{util}}(i)}, \quad (33)$$

where N_k, N_{tot} are the cluster bandit’s pre-update pull counts and $\bar{r}_k^{\text{prev}}$ is its mean reward *before* the current update. We use $(\lambda_u, \varepsilon) = (0.5, 10^{-8})$. The same reward formula is applied to the task-type bandit by replacing the cluster id $k(i)$ with the task-type id $\tau(i)$.

G Hyperparameters

Table 6 lists the hyperparameter values used throughout.

Table 6: EVE-Agent hyperparameters.	
Name	Value
λ_V (evidence-quality coefficient)	0.5
λ_B (brevity coefficient)	0.1
L_{max} (brevity cutoff)	256 tokens
m (verifier Monte Carlo budget)	5
λ_E (solver evidence-F1 coefficient)	0.3
Phase A HRPO steps	50
Phase B GRPO steps	50
Global batch size	256
Hop pmf $\{1, 2, 3, 4\}$	4:3:2:1
K_0 (initial cluster count)	10
α (granularity slope)	0.0 default
β (UCB exploration weight)	1.0
λ_u (selector utility weight)	0.5
ε (diversity smoothing)	10^{-8}

H Extended evidence-grounding diagnostics

Table 7 extends Table 1 of Section 4.2 by adding the per-benchmark answer-accuracy and evidence-presence columns. The evaluation uses the same 1,000 instances per benchmark, with 125 instances on Bamboo.

Two patterns confirm the bottleneck argument of Section 4.2. First, the prior system emits an evidence span in 90–99% of cases, comparable to the initial backbone, yet its judged evidence score in Table 1 is low. Second, the answer-and-evidence gap is much larger than the answer-only gap on the open-domain datasets: a verifier-trained solver is several times more likely to be simultaneously correct *and* supported.

Table 7: Full evidence-quality breakdown on 1,000 samples per benchmark (125 for Bamboogle). *Initial* is the Qwen2.5-3B-Instruct backbone; *Prior* is a faithful re-implementation of Yue et al. [2026]; *EVE-Agent* is the verifier-trained solver.

Benchmark	Initial (no s.)	Initial (s.)	Prior	EVE-Agent
<i>Answer accuracy</i>				
NQ	0.068	0.032	0.069	0.289
TriviaQA	0.219	0.125	0.257	0.437
PopQA	0.081	0.055	0.134	0.300
HotpotQA	0.057	0.029	0.110	0.209
2Wiki	0.034	0.013	0.077	0.176
MuSiQue	0.015	0.010	0.055	0.050
Bamboogle	0.200	0.024	0.104	0.088
<i>Evidence-present rate</i> (fraction of rollouts emitting an evidence span)				
NQ	0.996	0.785	0.991	0.999
TriviaQA	0.996	0.723	0.977	0.991
HotpotQA	0.996	0.567	0.971	0.998
2Wiki	0.996	0.555	0.933	0.997
MuSiQue	0.993	0.585	0.911	0.996